

透過 Google 助理動作與使用者互動

來源：<https://codelabs.developers.google.com/codelabs/actions-user-engagement?hl=zh-tw>

步驟數：6

瞭解如何運用這些功能持續吸引使用者採取行動。

目錄

1. [1. 總覽](#)
2. [2. 設定專案](#)
3. [3. 新增每日最新動態訂閱項目](#)
4. [4. 新增推播通知](#)
5. [5. 建立智慧助理連結](#)
6. [6. 後續步驟](#)

1. 總覽

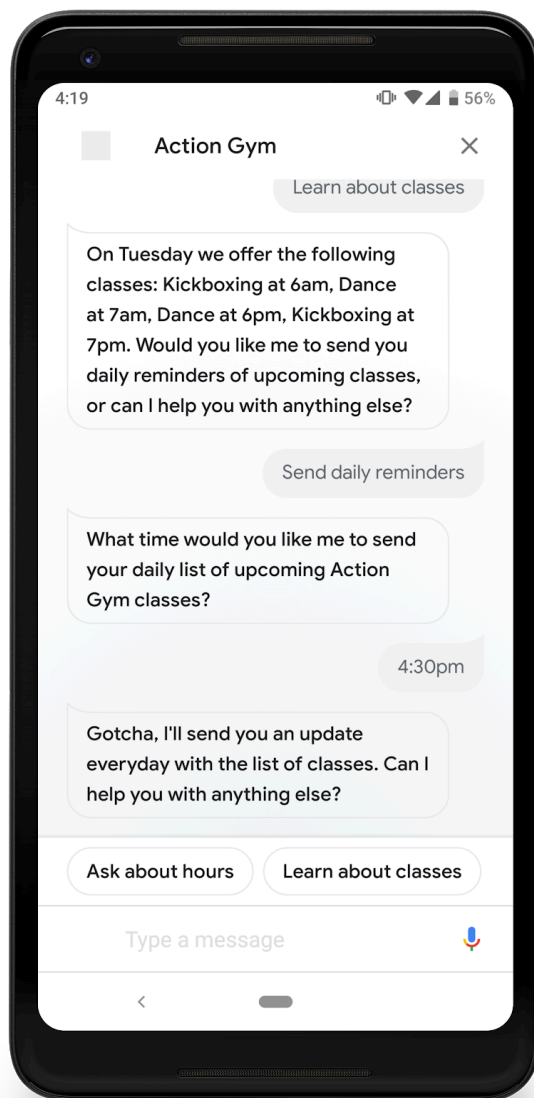
警告：對話動作將於 2023 年 6 月 13 日淘汰。詳情請參閱 [Conversational Actions 終止支援](#)。

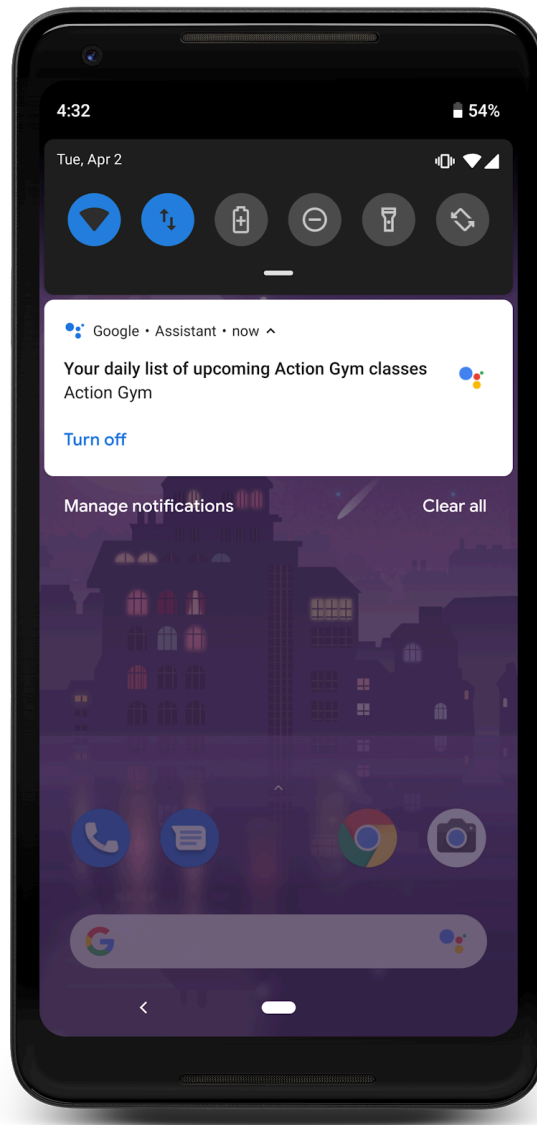
Actions on Google 是開發人員平台，可讓您建立軟體來擴充 [Google 助理](#) 的功能。Google 助理是虛擬個人助理，支援超過 10 億部裝置，包括智慧音箱、手機、汽車、電視、耳機等。使用者可以與 Google 助理對話，完成購物或預約車輛等工作。(如需完整清單，請參閱[動作目錄](#))。開發人員可以使用 Actions on Google，輕鬆建立及管理使用者與第三方服務之間令人愉悅且有效的對話體驗。

本程式碼研究室模組為進階內容，適合已具備 Google 助理 Actions 建構經驗的讀者。如果您沒有使用 Google 助理 Actions 開發的經驗，強烈建議您先完成入門程式碼研究室 ([第 1 級](#)、[第 2 級](#)和[第 3 級](#))，熟悉這個平台。這些進階課程將引導您瞭解一系列功能，協助您擴展動作的功能並增加目標對象。

評估動作成效的重要方式之一，是使用者參與度，也就是動作在使用者首次互動後，能否有效吸引他們回流。為簡化這項程序，您可以在 Action 中導入多項功能，讓使用者返回對話。

本程式碼研究室涵蓋 Google 助理動作的使用者參與度功能和最佳做法。





建構項目

您將強化已建構的功能，讓該功能能夠：

- 每天向使用者傳送更新，輕觸即可與動作對話
- 傳送推播通知給使用者，引導他們返回動作
- 建立連結，讓使用者透過行動網路瀏覽器前往你的動作

課程內容

- 什麼是使用者參與度？為何這項指標對動作的成功至關重要？
- 如何修改動作以提高使用者參與度
- 在不同類型的動作中使用哪些使用者參與度功能
- 如何使用 Actions API 透過 Google 助理傳送通知

軟硬體需求

您必須具備下列工具：

- 您偏好的 IDE/文字編輯器，例如 [WebStorm](#)、[Atom](#) 或 [Sublime](#)
- 終端機，用於執行已安裝 Node.js、npm 和 Git 的殼層指令

- 網路瀏覽器，例如 [Google Chrome](#)
- 本機開發環境，並安裝 Firebase 指令列介面
- 搭載 Google 助理的行動裝置 (Android 或 iOS)。(您必須使用與建立這個專案時相同的 Google 帳戶登入 Google 助理)。

強烈建議您熟悉 [JavaScript \(ES6\)](#)，雖然這並非必要條件，但有助於瞭解 Webhook 程式碼。

2. 設定專案

本節說明如何將使用者參與度功能新增至先前建構的完整動作。

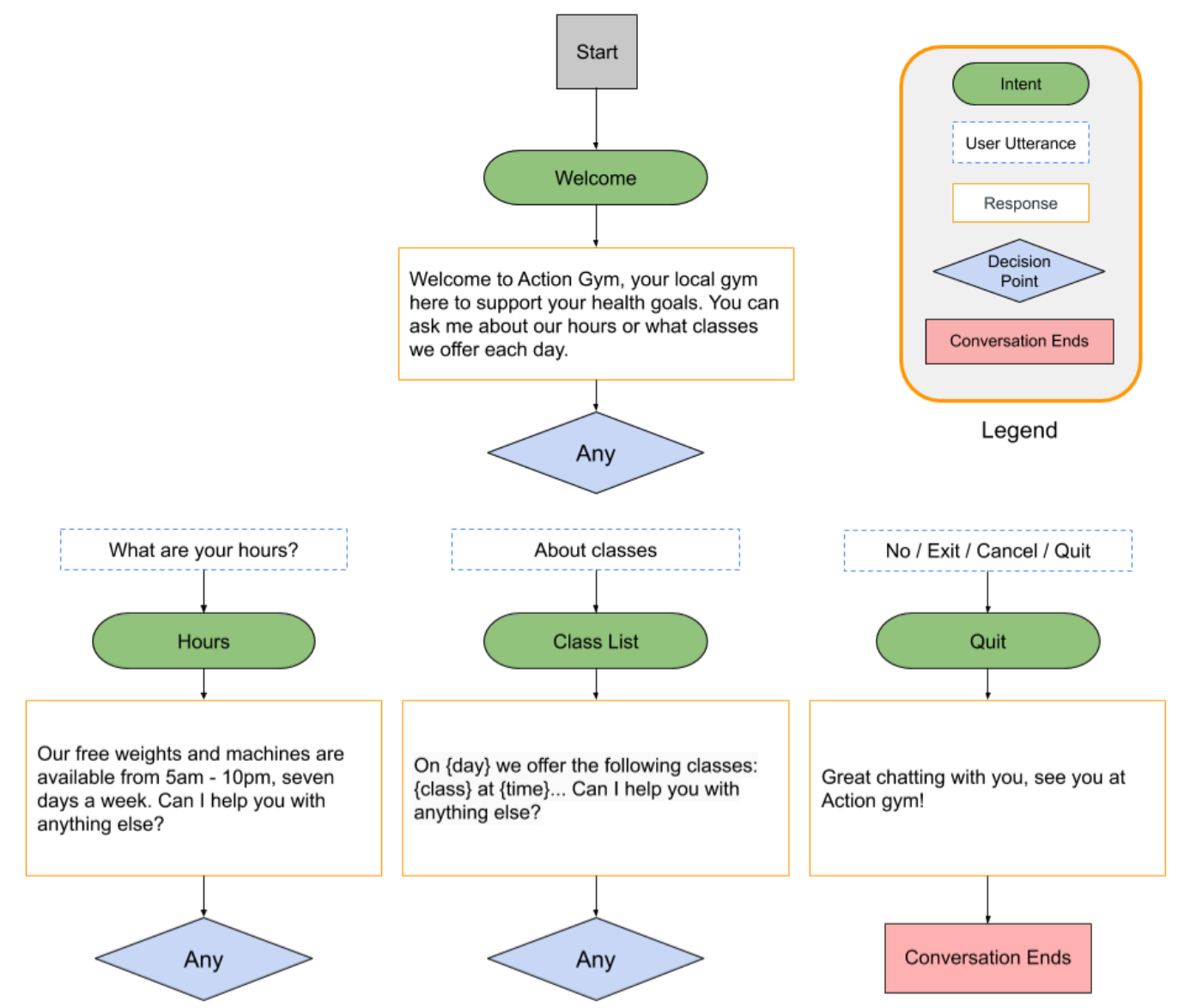
關鍵字詞：

- 使用者參與度：**這項指標用於評估使用者是否覺得產品或服務有價值。參與度可以透過返回動作的使用者人數來表示，這是用來判斷是否要宣傳動作的指標之一。高使用者參與度來自於動作的兩個部分：使用者日後會想查看的實用資訊，以及使用者最有可能擷取該資訊時，返回動作的方式。

瞭解範例

本程式碼研究室的範例是虛構健身房「Action Gym」的簡單動作。這項動作會提供健身房資訊，包括每日輪替的課程清單。這類資訊型動作很適合用於所有使用者參與度功能，因為輪流顯示的課程清單每天都會提供不同的實用資訊。

下圖顯示 Action Gym 範例的對話流程：



您將對對話方塊進行微幅修改，以更符合您新增的參與度功能。不過，對話的一般設計不會有太大變化。

下載基礎檔案

執行下列指令，複製本程式碼研究室的 GitHub 存放區：

```
git clone https://github.com/actions-on-google/user-engagement-codelab-nodejs
```

設定專案和代理程式

如要設定 Actions 專案和 Dialogflow 代理程式，請完成下列步驟：

1. 開啟 [Actions 管理中心](#)。
2. 按一下 [新專案]。
3. 輸入專案名稱，例如 `engagement-codelab`。
4. 按一下 [建立專案]。
5. 請勿選取類別，而是向下捲動至「更多選項」部分，然後按一下「對話」資訊卡。
6. 按一下「建立動作」展開選項，然後選取「新增動作」。
7. 按一下「新增第一個動作」。
8. 在「建立動作」對話方塊中，選取「自訂意圖」，然後按一下「建構」，啟動 Dialogflow 主控台。
9. 在 Dialogflow 主控台的代理程式建立頁面中，按一下「建立」。
10. 按一下左側導覽列中的



(齒輪圖示)。

11. 依序點選「匯出與匯入」和「從 Zip 檔案還原」。
12. 上傳您先前下載的 `/user-engagement-codelab-nodejs/start/` 目錄中的 `agent.zip` 檔案。
13. 輸入 `RESTORE`，然後按一下「還原」。
14. 按一下 [完成]。

部署執行要求

Actions 專案和 Dialogflow 代理程式都準備就緒後，請使用 Firebase Functions CLI 部署本機 `index.js` 檔案。

從基礎檔案複製的 `/user-engagement-codelab-nodejs/start/functions/` 目錄執行下列指令：

```
firebase use <PROJECT_ID>
```

提示：在 [Actions 控制台](#) 中，依序點選「總覽」>



(齒輪圖示) > 「專案設定」，即可找到動作的專案 ID。

```
npm install
```

```
firebase deploy
```

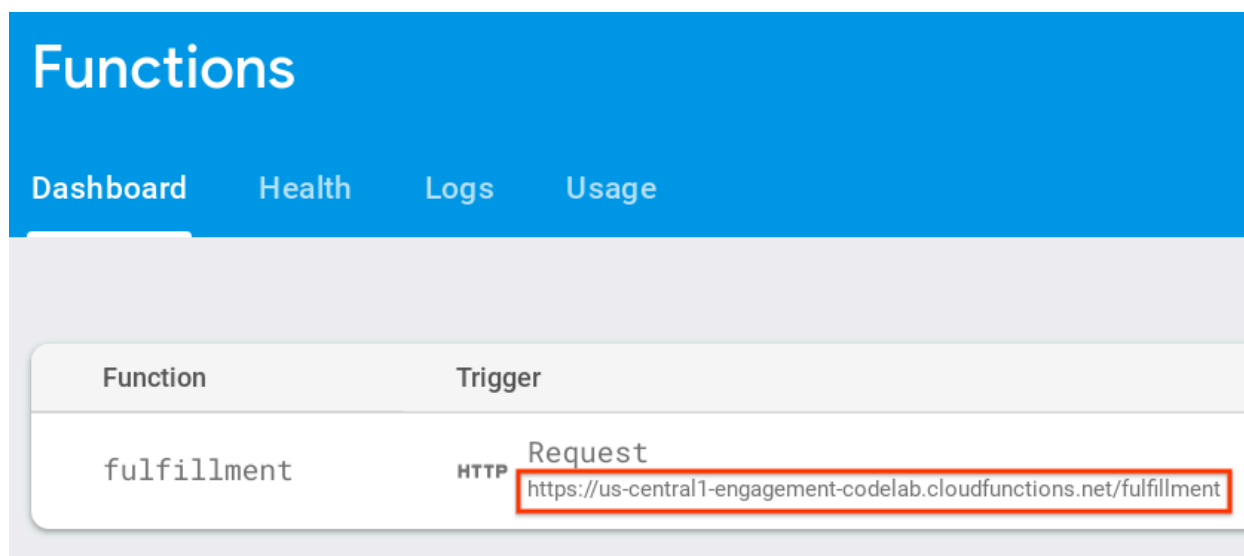
如果看到「An unexpected error has occurred from the CLI」錯誤訊息，請嘗試再次執行 `firebase deploy` 指令。

幾分鐘後，您應該會看到「**Deploy complete!**」，表示已成功將 Webhook 部署至 Firebase。

擷取部署網址

您必須向 Dialogflow 提供 Cloud 函式的網址。如要擷取這個網址，請按照下列步驟操作：

1. 開啟 [Firebase 控制台](#)。
2. 從選項清單中選取 Actions 專案。
3. 前往左側導覽列中的「開發」>「函式」。如果系統提示「選擇資料共用設定」，請按一下「稍後再執行」，忽略這個選項。
4. 在「資訊主頁」分頁下方，您應該會看到「履行」的項目，以及「觸發條件」下方的網址。請儲存這個網址，下一節會將其複製到 Dialogflow。



在 Dialogflow 中設定 Webhook 網址

現在您需要更新 Dialogflow 代理程式，以使用 Webhook 執行要求。步驟如下：

1. 開啟 [Dialogflow 控制台](#) (您可以視需要關閉 Firebase 控制台)。
2. 按一下左側導覽列中的「執行要求」。
3. 啟用 **Webhook**。
4. 如果網址尚未顯示，請貼上從 Firebase 資訊主頁複製的網址。
5. 按一下 [儲存]。

確認專案設定正確無誤

使用者應能叫用你的動作，取得有關 Action Gym 的資訊，包括營業時間的硬式編碼文字回覆，以及列出每週每日課程時間表的文字回覆。

提示：如要瞭解如何使用 Actions Simulator，請參閱[這份指南](#)。如果按照下列步驟操作時遇到任何問題，請參閱該頁面。

如要在 Actions Simulator 中測試動作，請按照下列步驟操作：

1. 在 Dialogflow 控制台左側導覽中，依序點選「Integrations」(整合) > 「Google Assistant」(Google 助理)。
2. 請確認已啟用「自動預覽變更」，然後點選「測試」，更新 Actions 專案。
3. 動作模擬器會載入動作專案。如要測試動作，請在「輸入」欄位中輸入 `Talk to my test app`，然後按下 Enter 鍵。
4. 畫面會顯示歡迎加入 Action Gym 的回應。請按照提示繼續對話，並確保每個輸入內容都有對應的回覆。

如果沒有看到這些回覆，可能是 Firebase 設定有問題。在這種情況下，請嘗試重複「部署服務」一節中的步驟。

Talk to my test app

 Sure. Here's the test version of my test app. Welcome to Action Gym, your local gym here to support your health goals. You can ask me about our hours or what classes we offer each day.

Ask about hours

 Our free weights and machines are available from 5am - 10pm, seven days a week. Can I help you with anything else?

Learn about classes

 On Thursday we offer the following classes: Kickboxing at 6am, Cycling at 7am, Dance at 6pm, Kickboxing at 7pm. Can I help you with anything else?

Quit 

 Great chatting with you!

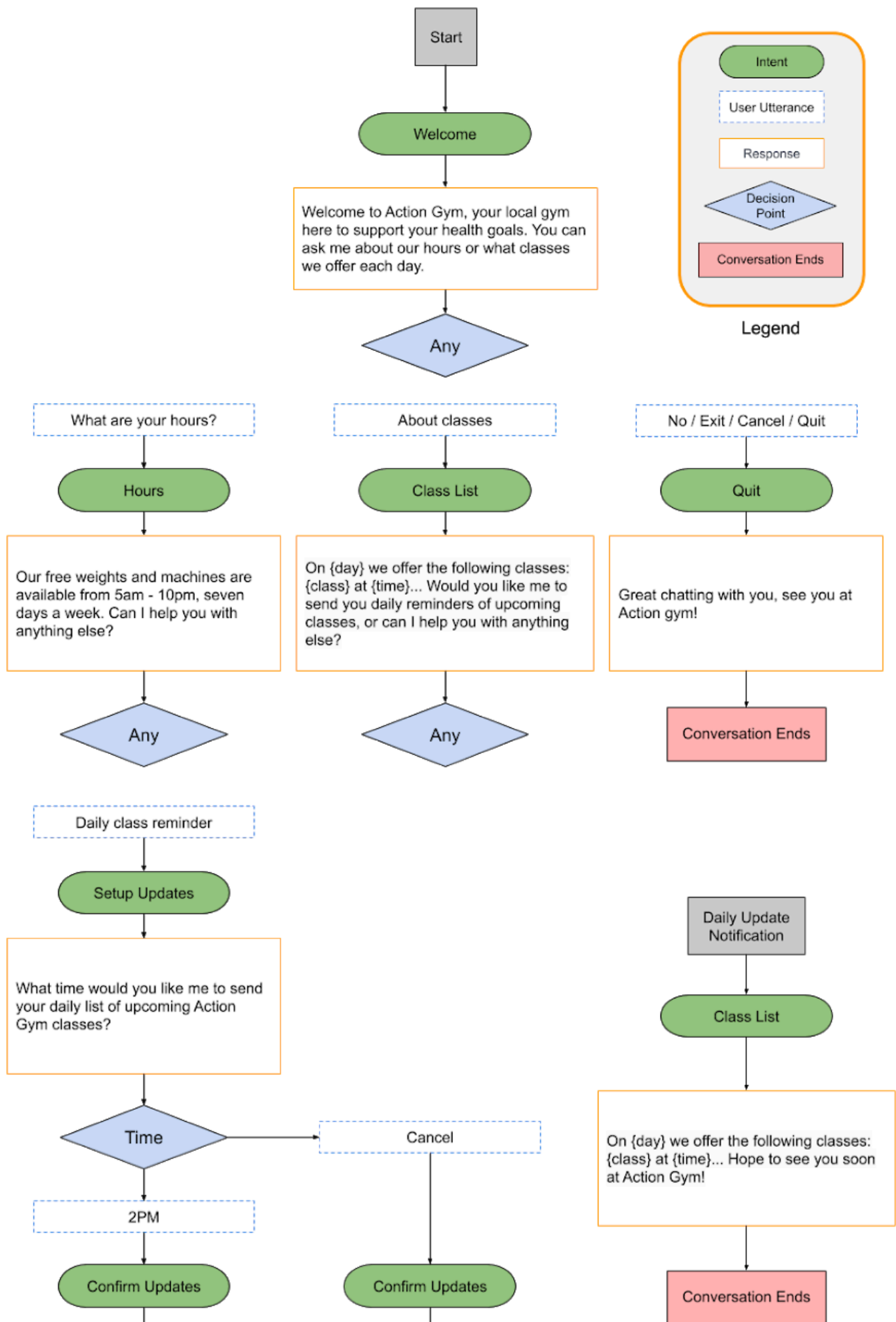
'my test app' left the conversation

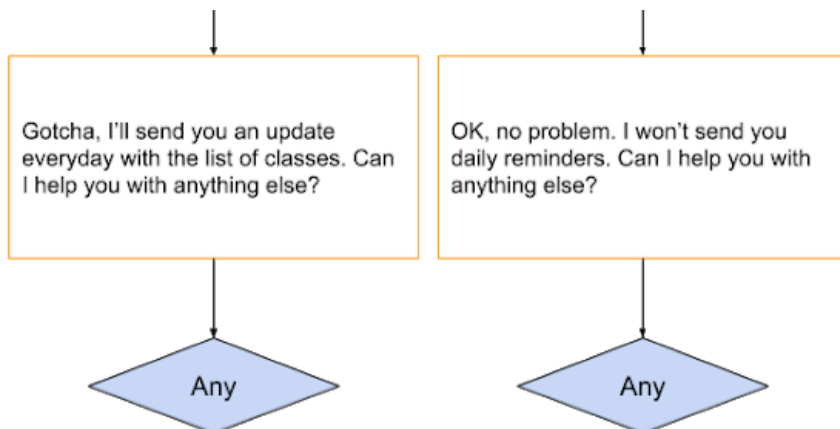
步驟 3 · 約 10 分鐘

3. 新增每日最新動態訂閱項目

吸引使用者的常見做法，是在最實用的時機提供資訊。方法是讓使用者選擇訂閱意圖的[每日更新](#)，系統會傳送 Google 助理通知，直接連結至該意圖的完成程序。

在這個步驟中，您將瞭解每日更新訂閱項目，並將這些項目新增至 Action 的 **Class List** 意圖。按照這些指示操作後，動作的對話會如下圖所示：





如何吸引使用者？

智慧型手機使用者可能很熟悉推播通知，這類通知會提供應用程式專屬資訊和最新動態。只要您傳送更新的意圖能持續為使用者帶來價值，每日更新訂閱功能就能輕鬆在 Google 助理以外的行動裝置上觸及使用。

每日更新是實用的參與度工具，但不一定需要納入每個動作。決定是否要在動作中新增每日更新訂閱時，請參考以下提示：

- 請確保每日更新能讓使用者每天看到不同的實用資訊。如果使用者每次輕觸每日更新都會看到相同的提示，可能過幾天就會取消訂閱。
- 如果使用者直接跳到每日更新意圖，請確保對話方塊對他們有意義。使用者不一定會從對話開頭開始，因此不應預期他們會有很多背景資訊。
- 先向使用者說明這項動作的好處，再提示他們訂閱每日更新。使用者看到訂閱選項時，應該會想「我每天都想看到這類內容」。
- 請勿重複建議使用者訂閱，以免造成困擾。在向使用者說明訂閱內容後，立即提供每日更新訂閱選項，並避免在其他地方不斷提醒他們。
- 觸發更新意圖後，請簡短對話。大多數每日更新應只包含單一回應，然後關閉，不需要使用者輸入內容。

提示：每日更新會以通知形式顯示在使用者裝置上，因此請參閱[設計實用通知](#)的最佳做法。每日更新通知的大部分內容都是為 Google 助理預先設計，但決定每日更新是否適合意圖或動作時，也適用這些原則。

開啟每日最新動態

您可以將每日更新訂閱項目新增至歡迎意圖，讓使用者從對話開始處加入，也可以新增至更具體的意圖，將使用者深層連結至對話中的某個位置。在本程式碼研究室中，**課程清單**意圖最為合適，因為對話內容每天都會變更，提醒使用者有哪些課程可選，或許會很有幫助。

如要為「課程清單」意圖啟用每日更新，請按照下列步驟操作：

1. 在 Actions 控制台中，按一下「開發」分頁，然後在左側導覽列中選擇「動作」。
2. 按一下「動作」清單下方的「課程名單」。
3. 在「使用者參與度」部分下方，切換「是否要為使用者提供每日更新」選項。
4. 設定描述每日更新內容的**內容標題**。系統會詢問「你想要我在什麼時候傳送每日『<內容標題>』給你？」，因此請務必使用描述性標題，並確認標題唸出來時聽起來正確。在本範例中，請將「內容標題」設為 `list of upcoming Action Gym classes`。
5. 按一下頁面頂端的「儲存」。

User engagement

Allow Google Assistant to send daily updates or push notifications to users, on behalf of your Action. [Learn more](#) 

Would you like to offer daily updates to users?  ☒

Would you like to let users add this Action to Google Assistant Routines? ☐

Would you like to send push notifications? If yes, user permission will be needed.  ☐

Content title (this will be shown to users) 

list of upcoming Action Gym classes

設定 Dialogflow

請按照下列步驟在 Dialogflow 主控台中，為每日更新訂閱流程建立意圖：

提示使用者訂閱

1. 設定新的意圖，處理使用者要求訂閱每日更新的內容。在 Dialogflow 主控台中，按一下左側導覽列中「Intents」(意圖) 旁邊的「+」按鈕，即可建立新意圖。
2. 將這個新意圖命名為 `Setup Updates`。
3. 在「訓練詞組」部分底下，新增下列使用者表達內容：
 - `Send daily reminders`
 - `Reminder`
 - `Remind me`
 - `Updates`
 - `Upcoming classes`
4. 在「Fulfillment」(執行要求) 部分下方，切換「Enable webhook call for this intent」(為這個意圖啟用 Webhook 呼叫) 選項。
5. 按一下頁面頂端的「儲存」。

• Setup Updates

SAVE

Contexts ?



Events ?



Training phrases ?

Search training phrases



” Add user expression

” upcoming classes

” updates

” remind me

” reminder

” Send daily reminders

Action and parameters



Responses ?



Fulfillment ?



☒ Enable webhook call for this intent

☐ Enable webhook call for slot filling

處理使用者的決定

1. 設定新的意圖，處理使用者對每日更新訂閱提示的回應。按一下左側導覽列中「意圖」旁的「+」按鈕，即可建立新意圖。
2. 將這個新意圖命名為 `Confirm Updates`。
3. 在「事件」部分下方，新增 `actions_intent_REGISTER_UPDATE`。無論使用者是否訂閱，只要完成每日最新消息訂閱流程，就會觸發這項 Dialogflow 事件。
4. 在「Fulfillment」（執行要求）部分下方，切換「Enable webhook call for this intent」（為這個意圖啟用 Webhook 呼叫）選項。
5. 按一下頁面頂端的「儲存」。

Confirm Updates

SAVE

Contexts ?



Events ?



actions_intent_REGISTER_UPDATE (X) Add event

Training phrases ?



Action and parameters



Responses ?



Fulfillment ?



☒ Enable webhook call for this intent

☐ Enable webhook call for slot filling

實作執行要求

如要在 Webhook 中實作執行要求，請完成下列步驟：

載入依附元件



在 `index.js` 檔案中，更新 `require()` 函式，從 `actions-on-google` 套件新增 `RegisterUpdate` 套件，讓匯入內容如下所示：

index.js

```
const {
  dialogflow,
  Suggestions,
  RegisterUpdate,
} = require('actions-on-google');
```

更新建議方塊



在 `index.js` 檔案中，將 `DAILY` 項目新增至建議事項晶片標題清單，讓 `Suggestion` 定義看起來像這樣：

index.js


```
// Suggestion chip titles
const Suggestion = {
  HOURS: 'Ask about hours',
  CLASSES: 'Learn about classes',
  DAILY: 'Send daily reminders',
};
```

為新意圖新增完成動作

使用者表示要訂閱時，請呼叫 `RegisterUpdate` 輔助程式，並提供更新的目標意圖 (**Class List**) 和類型 (`DAILY`)，啟動每日更新訂閱流程。訂閱流程完成後，Google 助理會觸發 `actions_intent_REGISTER_UPDATE` 事件，並提供 `status` 引數，說明訂閱是否成功。根據訂閱狀態，向使用者顯示不同的後續提示。



在 `index.js` 檔案中，新增下列程式碼：

index.js

```
// Start opt-in flow for daily updates
app.intent('Setup Updates', (conv) => {
  conv.ask(new RegisterUpdate({
    intent: 'Class List',
    frequency: 'DAILY',
  }));
});

// Confirm outcome of opt-in for daily updates
app.intent('Confirm Updates', (conv, params, registered) => {
  if (registered && registered.status === 'OK') {
    conv.ask(`Gotcha, I'll send you an update everyday with the ` +
      'list of classes. Can I help you with anything else?');
  } else {
    conv.ask(` I won't send you daily reminders. Can I help you with anything else?`);
  }
  if (conv.screen) {
    conv.ask(new Suggestions([Suggestion.HOURS, Suggestion.CLASSES]));
  }
});
```

為使用者提供其他提示

課程名單回覆會在結尾提供每日更新訂閱選項，但這會造成問題。由於使用者輕觸每日更新通知時也會觸發相同的回應，因此即使使用者剛收到每日更新通知，系統仍會要求他們訂閱。如何避免使用者認為自己需要重新訂閱？

幸好，`conv` 物件的引數包含使用者發起對話的位置資訊。您可以檢查 `conv` 引數是否含有 `UPDATES` 區段，判斷使用者是否從每日更新通知開始對話，並據此變更回覆內容。您也可以使用這個對話分支，在提供課程清單後立即關閉對話方塊，這符合我們每天更新內容時應簡短扼要的最佳做法。



在 `index.js` 檔案中，取代下列程式碼：

index.js

```
// Class list intent handler
app.intent('Class List', (conv, {day}) => {
  if (!day) {
    day = DAYS[new Date().getDay()];
  }
  const classes =
    [...new Set(schedule.days[day].map((d) => `${d.name} at ${d.startTime}`))]
    .join(', ');
  const classesMessage =
    `On ${day} we offer the following classes: ${classes}. ` +
    `Can I help you with anything else?`;
  conv.ask(classesMessage);
  if (conv.screen) {
    conv.ask(new Suggestions([Suggestion.HOURS]));
  }
});
```

改成下列内容：

index.js

```
// Class list intent handler
app.intent('Class List', (conv, {day}) => {
  if (!day) {
    day = DAYS[new Date().getDay()];
  }
  const classes =
    [...new Set(schedule.days[day].map((d) => `${d.name} at ${d.startTime}`))]
    .join(', ');
  let classesMessage = `On ${day} we offer the following classes: ${classes}. `;
  // If the user started the conversation from the context of a daily update,
  // the conv's arguments will contain an 'UPDATES' section.
  let engagement = conv.arguments.get('UPDATES');
  // Check the conv arguments to tailor the conversation based on the context.
  if (engagement) {
    classesMessage += `Hope to see you soon at Action Gym!`;
    conv.close(classesMessage);
  } else {
    classesMessage += `Would you like me to send you daily reminders of upcoming classes, or
can I help you with anything else?`;
    conv.ask(classesMessage);
    if (conv.screen) {
      conv.ask(new Suggestions([Suggestion.DAILY, Suggestion.HOURS]));
    }
  }
});
```

測試每日最新動態

在終端機中執行下列指令，將更新後的 Webhook 程式碼部署至 Firebase：

```
firebase deploy
```

提示：如要瞭解如何使用 Actions Simulator，請參閱[這份指南](#)。如果按照下列步驟操作時遇到任何問題，請參閱該頁面。

如要在動作模擬器中測試自訂重新提示，請按照下列步驟操作：

1. 在 Actions 控制台中，前往「測試」。
2. 在「Input」欄位中輸入 `Talk to my test app`，然後按下 Enter 鍵。
3. 輸入 `Learn about classes`，然後按下 Enter 鍵。現在，Action 的回覆應該會提供傳送每日提醒的選項。
4. 輸入 `Send daily reminders`，然後按下 Enter 鍵。
5. 輸入要查看更新的時間，然後按下 Enter 鍵。基於測試目的，請嘗試在目前時間的 3 到 5 分鐘後回覆。

Talk to my test app

 Okay. Getting the test version of my test app. Welcome to Action Gym, your local gym here to support your health goals. You can ask me about our hours or what classes we offer each day.

Learn about classes

 On Monday we offer the following classes: Yoga at 6am, Cycling at 7am, Dance at 6pm, Yoga at 7pm. Would you like me to send you daily reminders of upcoming classes, or can I help you with anything else?

Send daily reminders

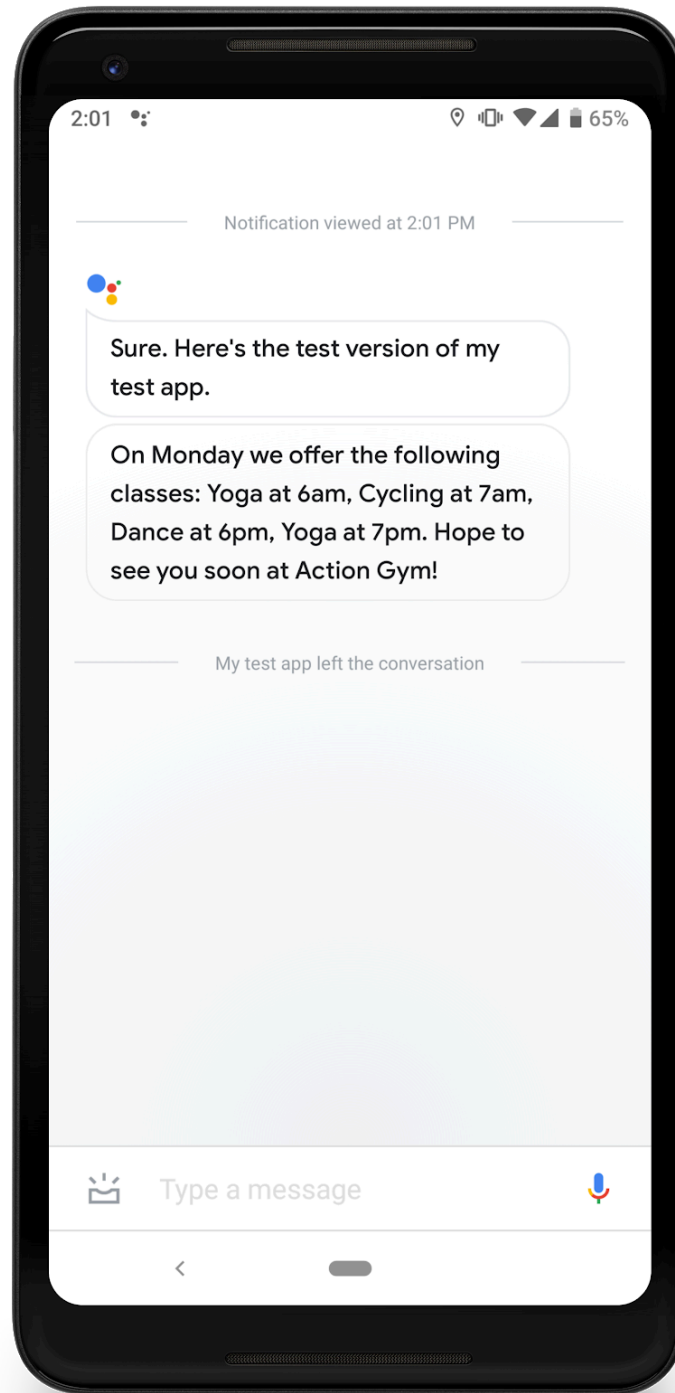
 What time would you like me to send your daily list of upcoming Action Gym classes?

2pm 

 Gotcha, I'll send you an update everyday with the list of classes. Can I help you with anything else?

如果在步驟 4 嘗試訂閱每日更新時發生錯誤，請返回「建構」>「動作」>「課程清單」，然後關閉每日更新。按一下「儲存」，重新開啟每日更新，再次按一下「儲存」，然後再次嘗試在模擬器中測試。

行動裝置會在您指定的時間收到 Google 助理的通知。請注意，這則通知可能要幾分鐘後才會顯示。輕觸通知後，系統應會透過深層連結直接開啟 Google 助理的「課程清單」意圖，並顯示即將開課的清單：



4. 新增推播通知

如要透過動作以外的方式與使用者互動，也可以呼叫 Actions API，向使用者傳送[推播通知](#)。與每日最新動態不同，Google 助理不會自動排定這類通知，因此你可以隨時傳送。

關鍵字詞：

- **Actions API**：Actions API 是 Google 助理 REST API，用於傳送通知給使用者。取得使用者授權後，您可以向 Actions API 端點傳送 `POST` 要求，系統會根據要求內容向使用者傳送推播通知。

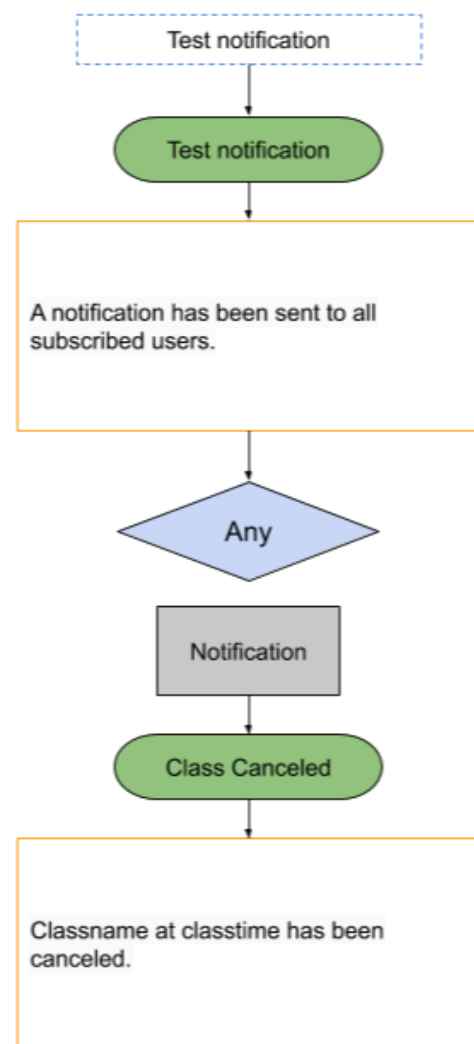
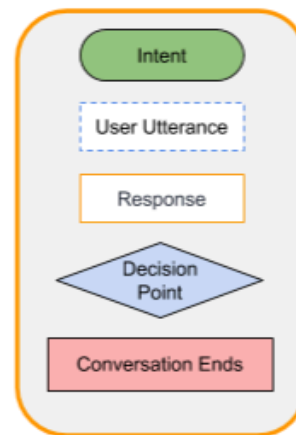
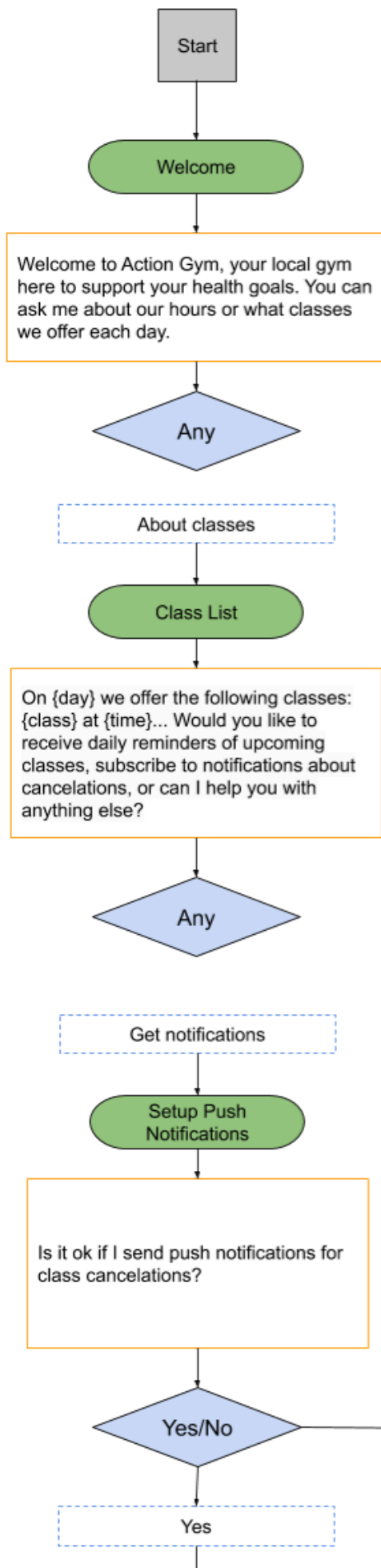
在這個步驟中，您將新增「Class Canceled」意圖，並傳送通知給使用者，告知課程已取消，藉此瞭解如何在動作中實作推播通知。您還需要設定下列三項元件，才能傳送通知：

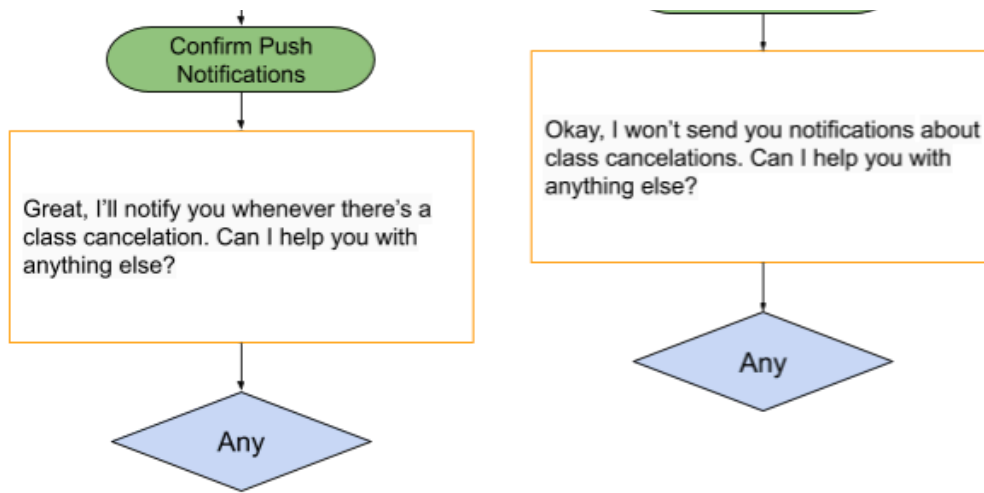
- **Actions API 帳戶**：您需要對 API 傳送 `POST` 要求，才能向使用者傳送通知，因此必須設定服務帳戶和憑證，才能與這個 API 互動。
- **權限輔助** - 您必須取得使用者授權，才能存取傳送推播通知所需的使用者 ID。在本範例中，您將使用用戶端程式庫函式呼叫權限輔助函式，並要求這個 ID。

關鍵字詞：

- **輔助意圖**：[輔助意圖](#)是一種意圖，當系統滿足這項意圖時，Google 助理會暫時接管對話，向使用者索取特定資訊。如果要求成功，輔助程式會將該資訊傳回給您。
- **儲存空間** - 如要向對話外的使用者傳送推播通知，您必須將使用者 ID 儲存在可隨時回呼的位置。在本範例中，您將設定 Firestore 資料庫，儲存每位使用者的資訊。

按照這些操作說明完成後，您會在 Action 的對話中新增下列對話方塊：





如何吸引使用者？

智慧型手機使用者可能很熟悉推播通知，這類通知會提供應用程式專屬資訊和最新動態。只要使用者有充分理由啟用推播通知，您就能透過這項功能在 Google 助理以外的行動裝置上與使用者互動。使用者已知道每天都會收到通知，但使用者無法判斷自己選擇接收的推播通知頻率，是偶爾收到通知，還是每天都會收到多則通知。

推播通知是實用的參與度工具，但不一定需要納入每個動作。決定是否要在動作中加入推播通知時，請參考以下提示：

- 為推播通知規劃一些範例時間表。如果每天只打算傳送一則推播通知，建議改用每日更新。
- 確保推播通知每次都能提供實用資訊。通知也可以深層連結至其中一個動作的意圖，因此請務必確保意圖實用且相關。
- 要求使用者訂閱推播通知時，請明確說明。他們應瞭解每則推播通知的內容，並大致知道通知的發送頻率。

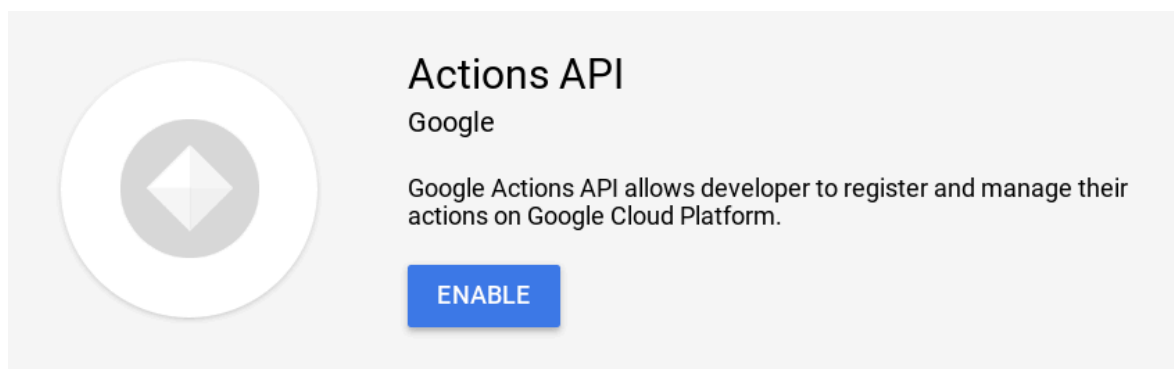
提示：請參閱[設計實用通知](#)的最佳做法。Google 助理的通知標誌和優先順序是預先決定的，但決定是否為動作新增推播通知時，也適用這些原則。

啟用 Actions API

1. 開啟 [Google Cloud 控制台](#)，然後在下拉式選單中選取 Actions 專案名稱。



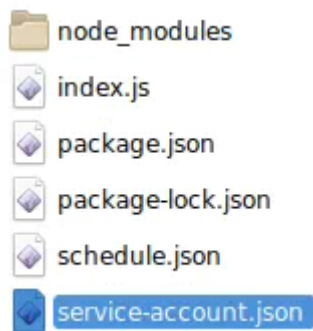
2. 在導覽選單 (≡) 中，依序前往「API 和服務」>「程式庫」。
3. 搜尋 **Actions API**，然後按一下「啟用」。



建立服務帳戶

Actions API 需要驗證，因此您必須建立服務帳戶才能傳送要求。請按照下列步驟，為 Actions API 建立及安裝服務帳戶金鑰：

1. 在 Google Cloud 控制台的導覽選單 (≡) 中，依序前往「API 和服務」>「憑證」。
2. 依序按一下「建立憑證」>「服務帳戶金鑰」。
3. 在「Service account」(服務帳戶) 下拉式選單中，選取「New Service Account」(新增服務帳戶)。
4. 填寫下列資訊：
 - 服務帳戶名稱：`service-account`
 - 角色：專案 > 擁有者
 - 服務帳戶 ID：`service-account` (一律會接在 @<project_id>.iam.gserviceaccount.com 後方)
 - 金鑰類型：JSON
5. 按一下「建立」。
6. 將下載的 JSON 檔案移至專案的 `/user-engagement-codelab/start/functions/` 目錄。
7. 將 JSON 檔案重新命名為 `service-account.json`。



啟用 Firestore

如要在對話以外傳送通知，您需要儲存使用者 ID，以便從通知程式碼中參照。在本範例中，我們使用 Firestore 資料庫儲存訂閱使用者的 ID。

請按照下列步驟為動作建立 Firestore 資料庫：

1. 在 [Firebase 控制台](#) 中，選取 Actions 專案名稱。
2. 在左側導覽選單中，依序前往「Develop」>「Database」，然後按一下「Create database」。
3. 選取「以測試模式啟動」。
4. 按一下「啟用」。

Security rules for Cloud Firestore



Once you have defined your data structure you will have to write rules to secure your data.

[Learn more](#)



Start in locked mode

Make your database private by denying all reads and writes



Start in test mode

Get set up quickly by allowing all reads and writes to your database

```
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write;
    }
  }
}
```



Anyone with your database reference will be able to read or write to your database

Enabling Cloud Firestore will preclude you from using Cloud Datastore with this project, notably from the associated App Engine app.

Cancel

Enable

提示：我們只會使用測試模式安全規則進行測試。您必須先制定更完善的資料安全規則，才能在正式版中發布動作。如要進一步瞭解 Cloud Firestore 安全性規則，請參閱 [Firestore 說明文件](#)。

設定 Dialogflow

請按照下列步驟在 Dialogflow 主控台中建立推播通知選擇加入流程：

提示使用者訂閱

- 設定新的意圖，處理使用者要求訂閱已取消課程的推播通知。在 Dialogflow 主控台中，按一下左側導覽列中「Intents」(意圖) 旁邊的「+」按鈕，即可建立新意圖。
- 將這個新意圖命名為「Setup Push Notifications」。
- 在「訓練詞組」部分底下，新增下列使用者表達內容：
 - Subscribe to notifications
 - Send notification
 - Notify me
 - Send class notifications
 - Cancelled notifications
- 在「Fulfillment」(執行要求) 部分下方，切換「Enable webhook call for this intent」(為這個意圖啟用 Webhook 呼叫) 選項。
- 按一下頁面頂端的「儲存」。

• Setup Push Notifications

SAVE



Contexts ?



Events ?



Training phrases ?

Search training phrases



” Add user expression

” Cancelled notifications

” Send class notifications

” Notify me

” Send notifications

” Subscribe to notifications

Action and parameters



Responses ?



Fulfillment ?



Enable webhook call for this intent



Enable webhook call for slot filling

處理使用者的決定

1. 設定新的意圖，處理使用者對推播通知訂閱提示的回應。按一下左側導覽列中「意圖」旁的「+」按鈕，即可建立新意圖。
2. 將這個新意圖命名為 `Confirm Push Notifications`。
3. 在「事件」部分下方，新增 `actions_intent_PERMISSION`。無論使用者是否訂閱，只要完成推播通知訂閱流程，就會觸發這個 Dialogflow 事件。
4. 在「Fulfillment」（執行要求）部分下方，切換「Enable webhook call for this intent」（為這個意圖啟用 Webhook 呼叫）選項。
5. 按一下頁面頂端的「儲存」。

• Confirm Push Notifications

SAVE

Contexts ?



Events ?



 Google Assistant Permission (X) Add event

Training phrases ?



Action and parameters



Responses ?



Fulfillment ?



☒ Enable webhook call for this intent

☐ Enable webhook call for slot filling

處理推播通知

您可以將推播通知連結至特定意圖，這樣使用者輕觸推播通知時，系統就會直接將他們深層連結至該意圖的動作。在本例中，請新增推送通知的意圖，提供取消課程的詳細資料。

如要新增意圖，讓使用者輕觸推播通知時觸發，請按照下列步驟操作：

1. 在 Dialogflow 主控台中，按一下左側導覽列中「Intents」(意圖) 旁邊的「+」按鈕，即可建立新意圖。
2. 將這個新意圖命名為「Class Canceled」。
3. 在「Training phrases」(訓練詞組) 部分底下，新增 **Cancellations** 做為使用者表達內容。
4. 在「Fulfillment」(執行要求) 部分下方，切換「Enable webhook call for this intent」(為這個意圖啟用 Webhook 呼叫) 選項。
5. 按一下頁面頂端的「儲存」。

• Class Canceled

SAVE

Contexts ?



Events ?



Training phrases ?

Search training phrases



” Add user expression

” cancelations

Action and parameters



Responses ?



Fulfillment ?



☒ Enable webhook call for this intent

☐ Enable webhook call for slot filling

在對話期間傳送測試通知

在實際工作環境中，您應使用與動作履行程式碼不同的指令碼傳送推播通知。在本範例中，請建立可叫用的意圖，以便在與動作交談時傳送推播通知。這個意圖僅供偵錯之用；實際上，不應由執行要求處理推播通知，也不應在 Action 的對話中觸發推播通知。

請按照下列步驟建立意圖，測試推播通知：

1. 為進行測試和偵錯，請設定新的意圖，以便向已訂閱的使用者傳送推播通知。在 Dialogflow 主控台中，按一下左側導覽列中「Intents」(意圖) 旁邊的「+」按鈕，即可建立新意圖。
2. 將這個新意圖命名為 `Test Notification`。
3. 在「Training phrases」(訓練詞組) 部分底下，新增 `Test notification` 做為**使用者表達內容**。
4. 在「Fulfillment」(執行要求) 部分下方，切換「Enable webhook call for this intent」(為這個意圖啟用 Webhook 呼叫) 選項。
5. 按一下頁面頂端的「儲存」。

• Test Notification

[SAVE](#)

Contexts ?



Events ?



Training phrases ?

Search training phrases



” Add user expression

” Test notification

Action and parameters



Responses ?



Fulfillment ?



Enable webhook call for this intent



Enable webhook call for slot filling

啟用推播通知

如要針對「Class Canceled」意圖啟用推播通知，請按照下列步驟操作：

1. 在 Dialogflow 控制台中，前往導覽列的「整合」。
2. 在「Google 助理」資訊卡上，按一下「整合設定」。
3. 將「Class Canceled」（課程已取消）新增為「Implicit invocation」（隱含叫用）意圖。Dialogflow 必須執行這個步驟，才能辨識使用者可透過「Class Canceled」（課程取消）意圖（輕觸推播通知）啟動對話。
4. 按一下 [關閉]。



Discovery

Explicit invocation *

Sign in required ?

Welcome



Specify the intent that is triggered when users request the app by name (for example "Ok Google, talk to Personal Chef."). [Learn more.](#)

Implicit invocation

Sign in required ?

Class List



Class Canceled



Add intent

Specify intents that trigger "deep-link" actions in your app, allowing users to invoke specific functionality, such as "OK Google, ask Personal Chef for a hot soup recipe". Providing good action phrases. [Learn more.](#)



Auto-preview changes



Dialogflow will propagate changes to the Actions Console and Assistant Simulator automatically.

5. 在 Actions 控制台中，按一下「開發」分頁，然後在左側導覽列中選擇「動作」。
6. 按一下「動作」清單下方的「課程已取消」。
7. 在「使用者參與度」部分下方，切換「是否要傳送推播通知？」選項。
8. 設定描述性**內容標題**，說明推播通知。系統會朗讀「我可以傳送『<內容標題>』的推播通知嗎？」，因此請務必確保標題資訊詳盡，且朗讀時聽起來正確。在本範例中，請將「內容標題」設為 `class cancelations`。
9. 按一下頁面頂端的「儲存」。

User engagement



Allow Google Assistant to send daily updates or push notifications to users, on behalf of your Action. [Learn more](#)

Would you like to offer daily updates to users? ? ☐

Would you like to let users add this Action to Google Assistant Routines? ☐

Would you like to send push notifications? If yes, user permission will be needed. ? ☒

Content title (this will be shown to users) ?

class cancellation

實作執行要求

如要在 Webhook 中實作執行要求，請完成下列步驟：

載入依附元件



在 `index.js` 檔案中，更新 `require()` 函式，從 `actions-on-google` 套件新增 `UpdatePermission` 套件，讓匯入內容如下所示：

index.js

```
const {
  dialogflow,
  Suggestions,
  RegisterUpdate,
  UpdatePermission,
} = require('actions-on-google');
```

更新建議方塊



在 `index.js` 檔案中，將 `NOTIFICATIONS` 項目新增至建議事項晶片標題清單，讓 `Suggestion` 定義看起來像這樣：

index.js

```
// Suggestion chip titles
const Suggestion = {
  HOURS: 'Ask about hours',
  CLASSES: 'Learn about classes',
  DAILY: 'Send daily reminders',
  NOTIFICATIONS: 'Get notifications',
};
```

設定新的匯入作業

如要連線至 Firestore 資料庫，請新增 `firebase-admin` 套件，並為儲存在資料庫中的欄位新增常數。此外，請匯入 `google-auth-library` 和 `request` 套件，以處理驗證和對 Actions API 的要求。



在 `index.js` 檔案中，將下列程式碼新增至匯入項目：

index.js

```
// Firebase admin import
const admin = require('firebase-admin');

// Initialize Firestore
admin.initializeApp();
const db = admin.firestore();

// Firestore constants
const FirestoreNames = {
  INTENT: 'intent',
  USER_ID: 'userId',
  USERS: 'users',
};

// Actions API authentication imports
const {auth} = require('google-auth-library');
const request = require('request');
```

提供設定課程取消通知的選項



在 `index.js` 檔案中，取代下列程式碼：

`index.js`


```
// Class list intent handler
app.intent('Class List', (conv, {day}) => {
  if (!day) {
    day = DAYS[new Date().getDay()];
  }
  const classes =
    [...new Set(schedule.days[day].map((d) => `${d.name} at ${d.startTime}`))]
    .join(', ');
  let classesMessage = `On ${day} we offer the following classes: ${classes}. `;
  // If the user started the conversation from the context of a daily update,
  // the conv's arguments will contain an 'UPDATES' section.
  let engagement = conv.arguments.get('UPDATES');
  // Check the conv arguments to tailor the conversation based on the context.
  if (engagement) {
    classesMessage += `Hope to see you soon at Action Gym!`;
    conv.close(classesMessage);
  } else {
    classesMessage += `Would you like me to send you daily reminders of upcoming classes, or
can I help you with anything else?`;
    conv.ask(classesMessage);
    if (conv.screen) {
      conv.ask(new Suggestions([Suggestion.DAILY, Suggestion.HOURS]));
    }
  };
});
});
```

改成下列內容：

index.js

```
// Class list intent handler
app.intent('Class List', (conv, {day}) => {
  if (!day) {
    day = DAYS[new Date().getDay()];
  }
  const classes =
    [...new Set(schedule.days[day].map((d) => `${d.name} at ${d.startTime}`))]
    .join(', ');
  let classesMessage = `On ${day} we offer the following classes: ${classes}. `;
  // If the user started the conversation from the context of a daily update,
  // the conv's arguments will contain an 'UPDATES' section.
  let engagement = conv.arguments.get('UPDATES');
  // Check the conv arguments to tailor the conversation based on the context.
  if (engagement) {
    classesMessage += `Hope to see you soon at Action Gym!`;
    conv.close(classesMessage);
  } else {
    classesMessage += `Would you like to receive daily reminders of upcoming classes,
    subscribe to notifications about cancelations, or can I help you with anything else?`;
    conv.ask(classesMessage);
    if (conv.screen) {
      conv.ask(new Suggestions([Suggestion.DAILY, Suggestion.NOTIFICATIONS,
      Suggestion.HOURS]));
    };
  };
});
```

為新意圖新增完成動作

如果使用者表示要訂閱推播通知，請呼叫 `UpdatePermission` 輔助函式，要求使用者授予權限。如果成功，`PERMISSION` 引數會新增至 `conv` 物件的引數，您可以檢查這些引數來調整對話。

取得使用者授權後，請從 `conv` 物件的引數中取得使用者 ID，並儲存至資料庫。稍後您會將這個使用者 ID 傳送至 Actions API，讓 Google 助理判斷通知的接收對象。

最後，為輕觸推播通知時觸發的 `Class Canceled` 意圖新增執行要求。在本範例中，回應是預留位置字串，但在適用於正式版的動作中，通知指令碼會提供更多動態資訊，說明取消的課程。



在 `index.js` 檔案中，新增下列程式碼：

index.js

```

// Call the User Information helper for permission to send push notifications
app.intent('Setup Push Notifications', (conv) => {
  conv.ask('Update permission for setting up push notifications');
  conv.ask(new UpdatePermission({intent: 'Class Canceled'}));
});

// Handle opt-in or rejection of push notifications
app.intent('Confirm Push Notifications', (conv) => {
  if (conv.arguments.get('PERMISSION')) {
    let userId = conv.arguments.get('UPDATES_USER_ID');
    if (!userId) {
      userId = conv.request.conversation.conversationId;
    }
    // Add the current conversation ID and the notification's
    // target intent to the Firestore database.
    return db.collection(FirestoreNames.USERS)
      .add({
        [FirestoreNames.INTENT]: 'Class Canceled',
        [FirestoreNames.USER_ID]: userId,
      })
      .then(() => {
        conv.ask(`Great, I'll notify you whenever there's a class cancelation. ` +
          'Can I help you with anything else?');
      });
  } else {
    conv.ask(`Okay, I won't send you notifications about class cancelations. ` +
      'Can I help you with anything else?');
  }
  if (conv.screen) {
    conv.ask(new Suggestions([Suggestion.CLASSES, Suggestion.HOURS]));
  }
});

// Intent triggered by tapping the push notification
app.intent('Class Canceled', (conv) => {
  conv.ask('Classname at classtime has been canceled.');
```

新增測試通知

如要傳送推播通知給使用者，請將 `POST` 要求傳送至 Actions API，並附上使用者 ID、通知標題和目標意圖。在本例中，觸發「測試通知」意圖會逐一檢查 Firestore 資料庫，並向所有已訂閱通知的使用者傳送推播通知。

請注意，在本範例中，您會在 Webhook 履行中加入傳送推播通知的程式碼，並在對話中叫用測試意圖，觸發該程式碼。在您打算發布的動作中，推送通知程式碼應位於與完成動作不同的指令碼中。



在 `index.js` 檔案中，新增下列程式碼：

index.js

```

// Debug intent to trigger a test push notification
app.intent('Test Notification', (conv) => {
  // Use the Actions API to send a Google Assistant push notification.
  let client = auth.fromJSON(require('./service-account.json'));
  client.scopes = ['https://www.googleapis.com/auth/actions.fulfillment.conversation'];
  let notification = {
    userNotification: {
      title: 'Test Notification from Action Gym',
    },
    target: {},
  };
  client.authorize((err, tokens) => {
    if (err) {
      throw new Error(`Auth error: ${err}`);
    }
    // Iterate through Firestore and send push notifications to every user
    // who's currently opted in to canceled class notifications.
    db.collection(FirestoreNames.USERS)
      .where(FirestoreNames.INTENT, '==', 'Class Canceled')
      .get()
      .then((querySnapshot) => {
        querySnapshot.forEach((user) => {
          notification.target = {
            userId: user.get(FirestoreNames.USER_ID),
            intent: user.get(FirestoreNames.INTENT),
          };
          request.post('https://actions.googleapis.com/v2/conversations:send', {
            'auth': {
              'bearer': tokens.access_token,
            },
            'json': true,
            'body': {'customPushMessage': notification, 'isInSandbox': true},
          }, (err, httpResponse, body) => {
            if (err) {
              throw new Error(`API request error: ${err}`);
            }
            console.log(`${httpResponse.statusCode}: ` +
              `${httpResponse.statusMessage}`);
            console.log(JSON.stringify(body));
          });
        });
      })
      .catch((error) => {
        throw new Error(`Firestore query error: ${error}`);
      });
  });
  conv.ask('A notification has been sent to all subscribed users.');
```

測試推播通知

在終端機中執行下列指令，將更新後的 Webhook 程式碼部署至 Firebase：

```
firebase deploy
```

提示：如要瞭解如何使用 Actions Simulator，請參閱[這份指南](#)。如果按照下列步驟操作時遇到任何問題，請參閱該頁面。

如要在動作模擬器中測試通知，請按照下列步驟操作：

1. 前往 Actions 控制台的「測試」分頁。
2. 在「Input」欄位中輸入 `Talk to my test app`，然後按下 Enter 鍵。
3. 輸入 `Learn about classes`，然後按下 Enter 鍵。
4. 輸入 `Get notifications`，然後按下 Enter 鍵。
5. 如果尚未授予動作傳送推播通知的權限，請輸入 `yes` 並按下 Enter 鍵。
6. 輸入 `yes`，然後按下 Enter 鍵。您的 Google 帳戶現在應該已訂閱這項動作的推播通知。

Talk to my test app



Sure. Getting the test version of my test app. Welcome to Action Gym, your local gym here to support your health goals. You can ask me about our hours or what classes we offer each day.

Learn about classes



On Monday we offer the following classes: Yoga at 6am, Cycling at 7am, Dance at 6pm, Yoga at 7pm. Would you like to receive daily reminders of upcoming classes, subscribe to notifications about cancelations, or can I help you with anything else?

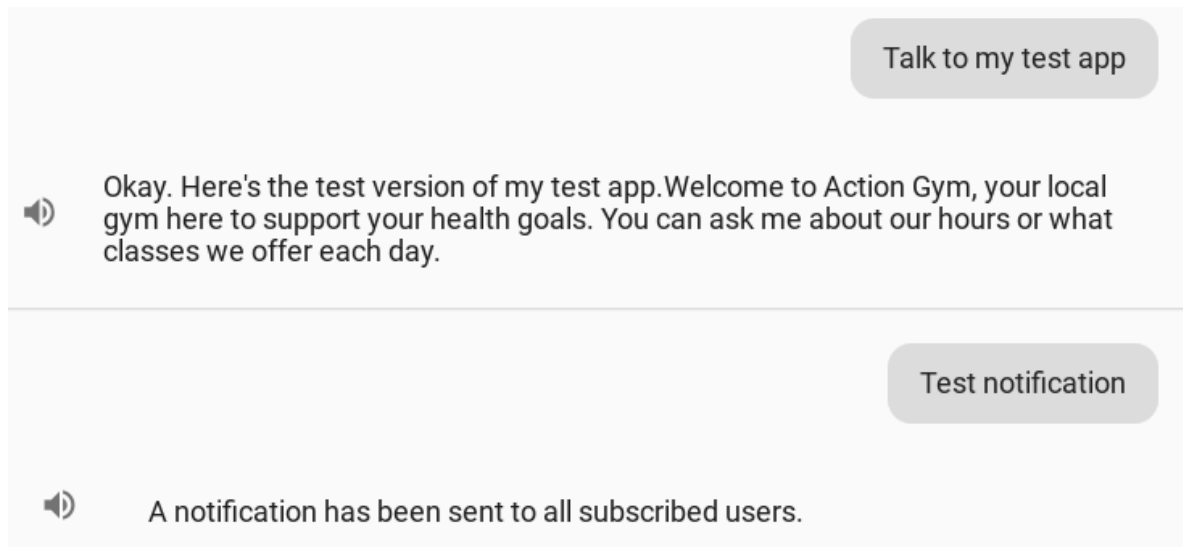
Get notifications



Great, I'll notify you whenever there's a class cancelation. Can I help you with anything else?

如果在步驟 4 嘗試訂閱推播通知時發生錯誤，請返回「Build」>「Actions」>「Class Canceled」，然後關閉推播通知。按一下「Save」，重新開啟推播通知，再次按一下「Save」，然後再次嘗試在模擬器中測試。

7. 輸入 `no` 並按下 Enter 鍵即可退出。
8. 輸入 `Talk to my test app` 並按下 Enter 鍵，即可開始新對話。
9. 輸入 `Test notification`，然後按下 Enter 鍵。



幾分鐘後，行動裝置上應該會收到「Test Notification from Action Gym」的 Google 助理推播通知。輕觸這則通知會深層連結至動作的「課程已取消」意圖。

如果無法順利收到通知，請按照手機上的測試說明操作。在手機上開啟 Google 助理，然後說出「`talk to my test app`」，接著按照上述測試說明從步驟 3 開始操作。



5. 建立智慧助理連結

我們目前已討論過可實作的參與度功能，讓使用者持續返回你的動作，但前提是使用者會探索及使用你的動作。

您可以建立[Google 助理連結](#)，將行動裝置使用者直接帶往 Google 助理上的動作。由於 Google 助理連結是標準超連結，因此您可以將其加入網站或任何網路行銷素材，例如網誌或社群媒體貼文。

在本步驟中，您將瞭解 Google 助理連結的用途、如何為動作的歡迎意圖建立連結，以及如何將連結新增至簡單的網站進行測試。

如何吸引使用者？

吸引使用者首次使用你的動作可能很困難，尤其是當他們需要在 Google 助理上明確叫用你的動作時。Google 助理連結可提供動作的直接連結，減少這類阻礙。使用者在支援 Google 助理的裝置上點選 Google 助理連結後，系統會直接將他們帶往你的動作。如果使用者在非行動裝置或任何不支援 Google 助理的裝置上開啟連結，系統仍會將他們帶往[動作目錄](#)資訊 (如果已發布)，因此連結仍可向這些使用者宣傳動作。

Google 助理連結是實用的參與度工具，因此如果您打算透過網站或社群媒體宣傳動作，建議建立這類連結。建立及發布 Google 助理連結前，請先參考下列提示：

- 只有在發布動作後，Google 助理連結才會生效。專案處於草稿狀態時，連結只能在您自己的裝置上運作。其他人則會前往「動作」目錄中的 404 頁面。
- 您可以在[Alpha 或 Beta 版環境](#)中發布動作，讓使用者在發布前測試 Google 助理連結。請注意，只有參與 Alpha 版或 Beta 版測試的使用者，才能測試 Google 助理連結。
- 請確保 Google 助理連結的目標意圖能為新使用者留下好印象。歡迎意圖是 Google 助理連結的預設目的地，因為它應該已能妥善介紹你的動作。

提示：在本範例中，您只會建立簡單的 Google 助理連結，將使用者導向歡迎意圖，但您可以進行更多操作，自訂 Google 助理連結。您可以在連結中附加網址參數，將使用者導向特定意圖，甚至加入意圖參數，自訂使用者點選連結後收到的回覆。雖然我們不會在本程式碼研究室中介紹這些進階選項，但您可以在[開發人員說明文件](#)中進一步瞭解。

開啟 Google 助理連結

請按照下列步驟，為歡迎意圖建立 Google 助理連結：

- 在 Actions 控制台中，按一下「開發」分頁，然後在左側導覽列中選擇「動作」。
- 按一下「動作」清單下方的「actions.intent.MAIN」。
- 在「連結」部分下方，切換「是否要為這項動作啟用網址」選項。
- 設定可描述動作的**連結標題**。請使用簡單的動詞和名詞組合做為名稱，說明使用者可透過動作完成哪些事項。在本例中，請將「連結標題」設為 `learn about Action Gym`。
- 複製本頁底部的 HTML 程式碼片段，並儲存以供稍後使用。
- 按一下頁面頂端的「儲存」。

Links

Enable a URL for your Action, so that users can discover and directly engage with it (e.g., reference the URL from your website).

Would you like to enable a URL for this Action? ☒

Link title (Required) ?

learn about Action Gym

Copy the snippet below and use it in your code

```
<a href="https://assistant.google.com/services/invoke/uid/000000efb5f2fd97">  
  Ask my test app to learn about Action Gym  
</a>
```



部署測試網站

如要測試 Google 助理連結，可以使用 Firebase 工具，在部署執行要求時一併部署測試網站。我們已為這個範例建構簡單的測試網站，您只需要新增 Google 助理連結即可。

前往完成項目的 `/user-engagement-codelab-nodejs/start/public/` 目錄，並在文字編輯器中開啟 `index.html` 檔案。



在 `index.html` 檔案中，將 Google 助理連結的 HTML 片段貼到 `body` 元素中。檔案最後應如下方程式碼片段所示：

index.html

```
<body>  
  <p>  
    <a href="https://assistant.google.com/services/invoke/uid/000000efb5f2fd97">Ask my  
test app to learn about Action Gym  
  </a>  
  </p>  
</body>
```

測試 Google 助理連結

在終端機中執行下列指令，將測試網站部署至 Firebase：

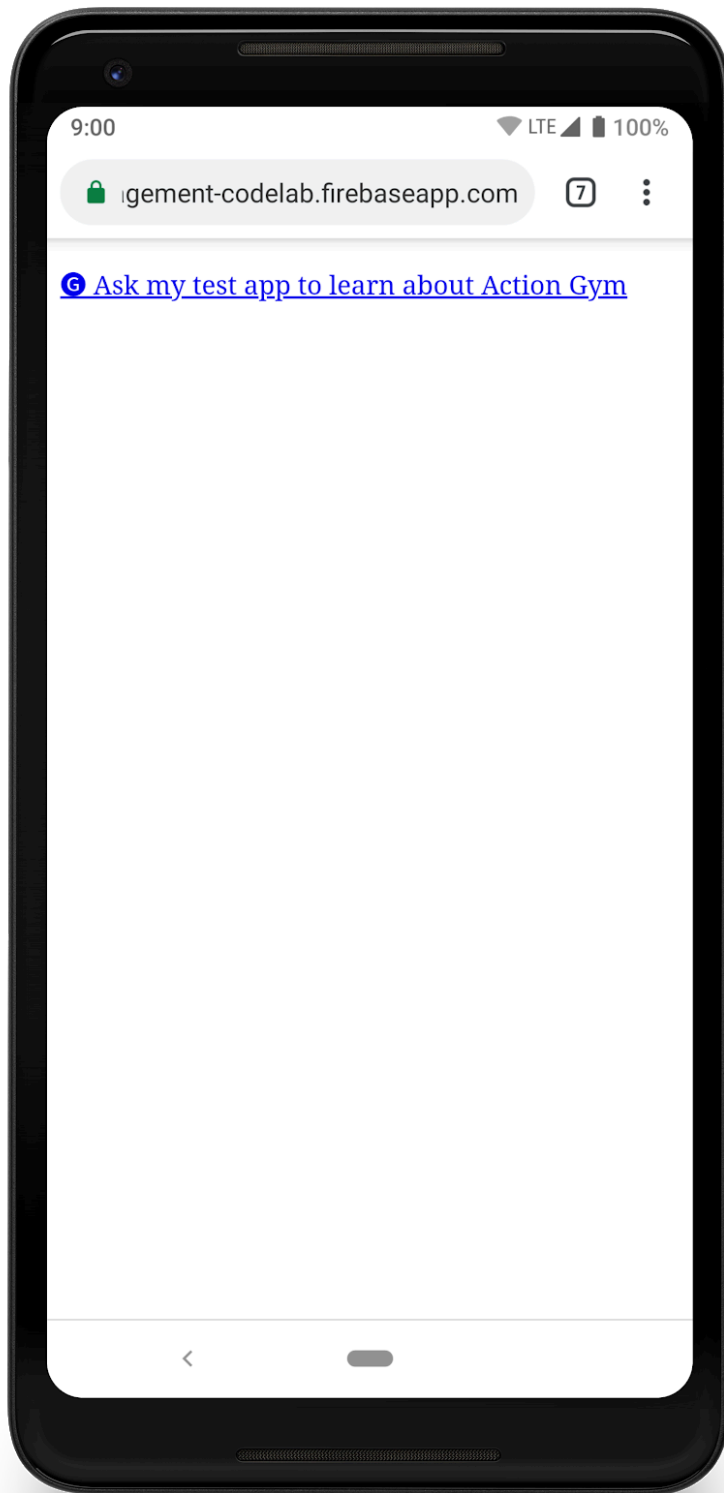
```
firebase deploy
```

部署指令執行完畢後，請記下輸出內容中的「託管網址」。

✔ **Deploy complete!**

Project Console: <https://console.firebase.google.com/project/engagement-codelab/overview>
Hosting URL: <https://engagement-codelab.firebaseio.com>

在行動裝置的網路瀏覽器中前往這個網址，您應該會在測試網站上看到 Google 助理連結。在行動裝置上點選這個連結後，應該會前往 Google 助理中 Action 的歡迎意圖。



你也可以在電腦瀏覽器上前往主機網址，由於你的動作尚未發布，因此應該會前往[Google 助理目錄](#)中的 404 頁面。

6. 後續步驟

恭喜！

您現在已瞭解開發動作時使用者參與度的重要性、平台提供的使用者參與度功能，以及如何將各項功能新增至動作。

其他學習資源

如要進一步瞭解動作的使用者參與度，請參閱下列資源：

- [使用者參與度](#)和 [Google 助理連結說明文件](#)：官方 Actions on Google 說明文件，說明本程式碼研究室討論的功能和一般使用者參與度。
- [使用者留存率分析](#)：Actions 控制台的分析功能文件，可顯示已發布動作的使用者留存率。
- [對話式設計指南](#)：設計使用者參與度功能的最佳做法和指南。
- [Actions on Google GitHub 存放區](#)：範例程式碼和程式庫。
- [r/GoogleAssistantDev](#)：專為使用 Google 助理開設的官方 Reddit 社群。

在 Twitter 上追蹤 [@ActionsOnGoogle](#)，隨時掌握最新公告，並在推文中加上 [#AoGDevs](#)，分享您開發的內容！

意見回饋問卷調查

離開前，請[填寫這份表單](#)，告訴我們你的想法！
